

CHAPTER 08

영상의 기하학적 변환

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

8장 영상의 기하학적 변환

8.1 어파인 변환

8.2 투시 변환

8.1 어파인 변환

회전 변환 (rotation transformation)

- 특정 좌표를 기준으로 영상을 원하는 각도만큼 회전하는 변환임
- 원점을 기준으로 영상을 반시계 방향으로 θ 만큼 회전하는 변환을 그림 8-10에 나타냄
- 그림 8-10에서 노란색 사각형은 원본 영상이고, 녹색으로 표시한 사각형이 회전 변환으로 생성된 결과 영상임

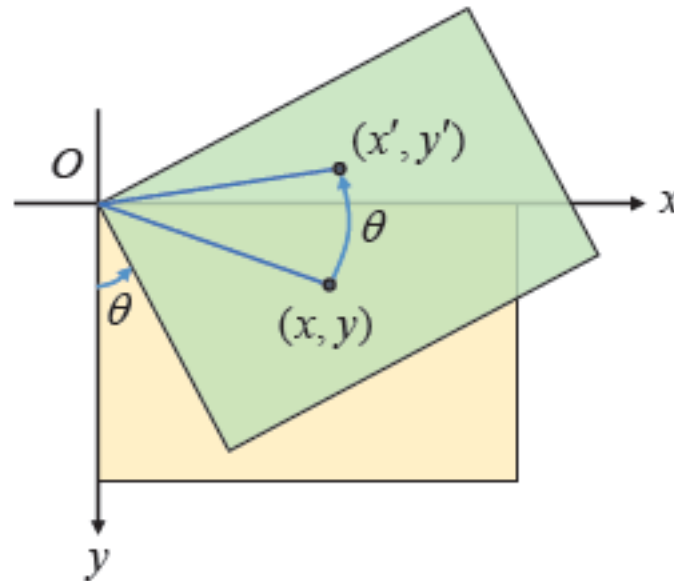


그림 8-10 영상의 회전 변환

회전 변환

- 원점을 기준으로 영상의 회전 변환에 의해 입력 영상의 점 (x, y) 가 이동하는 점의 좌표 (x', y') 는 다음과 같이 삼각함수를 이용하여 구할 수 있음 -> 어파인 변환임

$$\begin{cases} x' = \cos \theta \cdot x + \sin \theta \cdot y \\ y' = -\sin \theta \cdot x + \cos \theta \cdot y \end{cases} \text{ 또는 } \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$= \begin{pmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

회전 변환

- 원점을 기준으로 영상을 반시계 방향으로 θ 만큼 회전하는 어파인 변환 행렬 M 은 다음과 같이 정의됨

$$M = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \end{bmatrix}$$

- $\cos()$ 함수와 $\sin()$ 함수를 이용하여 앞과 같은 행렬을 생성하고, $\text{warpAffine}()$ 함수를 사용하면 영상을 회전시킬 수 있음
- 다만 영상을 회전하는 경우가 실제로 많이 발생하기 때문에 OpenCV는 영상의 회전을 위한 어파인 변환 행렬을 생성하는 $\text{getRotationMatrix2D}()$ 함수를 제공함

회전 변환

```
Mat getRotationMatrix2D(Point2f center, double angle, double scale);
```

- **center** 회전 중심 좌표
- **angle** 회전 각도. 양수는 반시계 방향, 음수는 시계 방향을 의미합니다. (단위: 도)
- **scale** 회전 후 추가적으로 확대 또는 축소할 비율. 크기 변환은 수행하지 않으려면 1을 지정합니다.
- **반환값** 2x3 어파인 변환 행렬(CV_64F)

- 영상을 원점이 아닌 특정 좌표를 기준으로 회전시키거나 필요한 경우 크기 변환(**회전 중심을 기준으로 확대, 축소됨**)까지 함께 수행하는 어파인 변환 행렬을 쉽게 계산
- getRotationMatrix2D() 함수가 반환하는 어파인 변환 행렬은 다음과 같이 계산됨

$$\begin{bmatrix} \alpha & \beta & (1-\alpha) \cdot \text{center.x} - \beta \cdot \text{center.x} \\ -\beta & \alpha & \beta \cdot \text{center.x} + (1-\alpha) \cdot \text{center.x} \end{bmatrix}$$

- 이 수식에서 $\alpha = \text{scale} \cdot \cos(\text{angle})$, $\beta = \text{scale} \cdot \sin(\text{angle})$ 을 의미함

코드8-5: 회전 변환

```
#include <iostream>
using namespace std;
#include <opencv2/opencv.hpp>
using namespace cv;
int main(void)
{
    Mat src = imread("tekapo.bmp");
    if (src.empty()) {cerr << "Image load failed!" << endl; return -1;}

    Point2f cp(src.cols/2.f, src.rows/2.f);
    Mat M = getRotationMatrix2D(cp, 20, 1);
    Mat dst;
    warpAffine(src, dst, M, Size());

    imshow("src", src);
    imshow("dst", dst);
    waitKey();
    return 0;
}
```


코드8-5: 회전 변환

- 영상의 중심 좌표를 Point2f 자료형의 변수 cp에 저장하고, 이 점을 기준으로 영상을 반시계 방향으로 20°만큼 회전함
- 입력 영상의 일부가 잘리지 않게 영상을 회전하려면 결과 영상의 크기를 더 크게 설정하고 회전과 이동 변환을 함께 고려해야 함



그림 8-11 영상의 회전 변환 예제 실행 결과

회전 변환

```
void rotate(InputArray src, OutputArray dst, int rotateCode);
```

- `src` 입력 행렬
- `dst` 출력 행렬
- `rotateCode` 회전 각도 지정 플래그, `ROTATE_90_CLOCKWISE`, `ROTATE_180`, `ROTATE_90_COUNTERCLOCKWISE` 세 개의 상수 중 하나를 지정합니다.

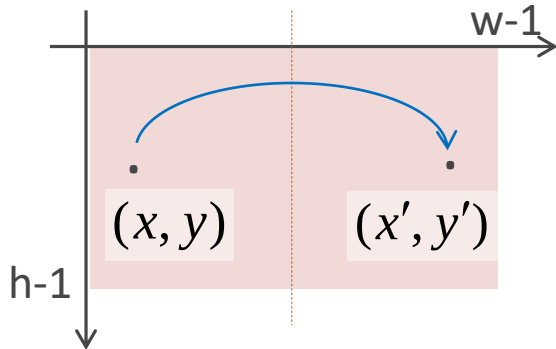
- 영상을 90° 단위로 회전하고 싶은 경우에는 `rotate()` 함수를 사용할 수 있음

대칭 변환

- 영상의 기하학적 변환 중에 영상의 좌우를 서로 바꾸거나, 또는 상하를 뒤집는 형태의 변환도 있음
- 영상을 마치 거울에 비친 것처럼 좌우를 바꾸는 변환을 좌우 대칭 또는 좌우 반전이라고 함
- 영상의 상하를 뒤집은 것처럼 바꾸는 변환을 상하 대칭 또는 상하 반전이라고 함
- 대칭 변환은 입력 영상과 같은 크기의 결과 영상을 생성함
- 입력 영상의 픽셀과 결과 영상의 픽셀이 일대일로 대응되므로 보간법이 필요하지 않음

대칭 변환

- 영상의 좌우 대칭 변환에 의한 좌표 변환 수식은 다음과 같음



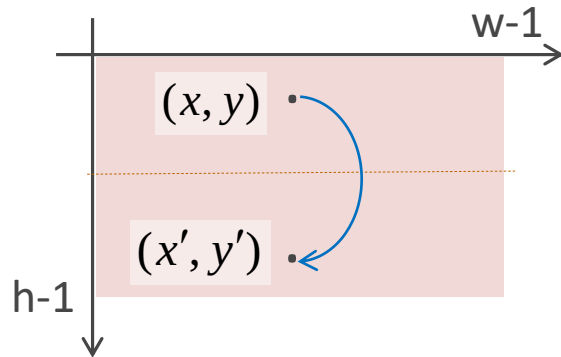
$$\begin{cases} x' = w - 1 - x \\ y' = y \end{cases}$$

- 이 수식에서 (x, y) 는 입력 영상의 픽셀 좌표이고, (x', y') 는 결과 영상의 픽셀 좌표이며, **w 는 입력 영상의 가로 크기임**
- 앞 수식을 행렬 형태로 쓰면 다음과 같음 -> 어파인 변환의 일종임

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} w-1 \\ 0 \end{bmatrix} = \begin{pmatrix} -1 & 0 & w-1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

대칭 변환

- 영상의 상하 대칭 변환을 수식으로 정리하면 다음과 같음



$$\begin{cases} x' = x \\ y' = h - 1 - y \end{cases}$$

- 수식에서 **h** 는 입력 영상의 세로 크기를 의미함
- 행렬 형태로 쓰면 다음과 같음 -> 어파인 변환의 일종임

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 0 \\ h-1 \end{bmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & h-1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

대칭 변환

```
void flip(InputArray src, OutputArray dst, int flipCode);
```

- **src** 입력 영상
- **dst** 결과 영상. src와 같은 크기, 타입
- **flipCode** 대칭 방법 지정 플래그. flipCode가 양수이면 좌우 대칭, 0이면 상하 대칭, 음수이면 상하 대칭과 좌우 대칭을 모두 수행합니다.

- flipCode -> 1 : 좌우대칭
- flipCode -> 0 : 상하대칭
- flipCode -> -1 : 상하+좌우대칭

코드8-6: 대칭 변환

```
// 생략
int main(void)
{
    Mat src = imread("eastsea.bmp");
    if (src.empty()) {cerr << "Image load failed!" << endl;return -1;}
    imshow("src", src);

    Mat dst;
    int flipCode[] = { 1, 0, -1 };
    for (int i = 0; i < 3; i++) {
        flip(src, dst, flipCode[i]);
        String desc = format("flipCode: %d", flipCode[i]);
        putText(dst, desc, Point(10, 30), FONT_HERSHEY_SIMPLEX, 1.0,
                               Scalar(255, 0, 0), 1, LINE_AA);

        imshow("dst", dst);
        waitKey();
    }
    return 0;
}
```

코드8-6: 대칭 변환



(a)



(b)

그림 8-12 영상의 대칭 변환 예제 실행 결과

코드8-6: 대칭 변환



(c)



(d)

그림 8-12 영상의 대칭 변환 예제 실행 결과

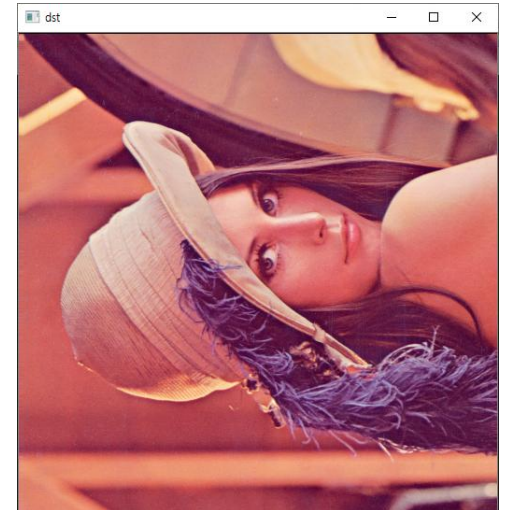
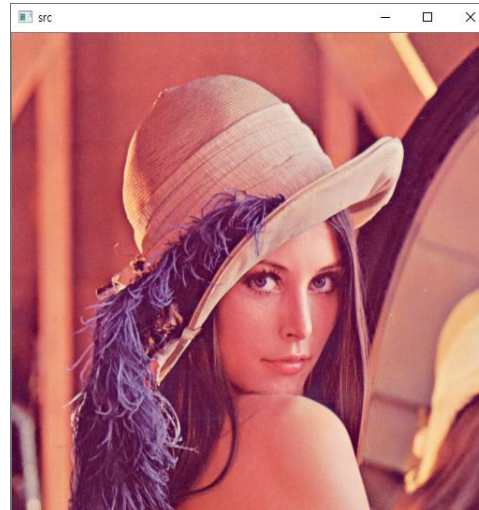
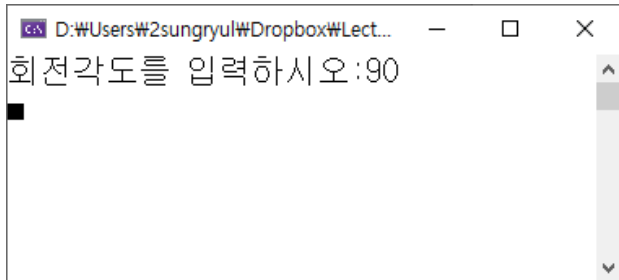
실습과제1

- 출력영상이 입력영상보다 2배 크고 회전은 입력 영상의 가운데 좌표를 기준으로 시계방향으로 45도 회전하도록 프로그램을 작성하시오. 크기변환과 회전변환을 모두 이용하시오.



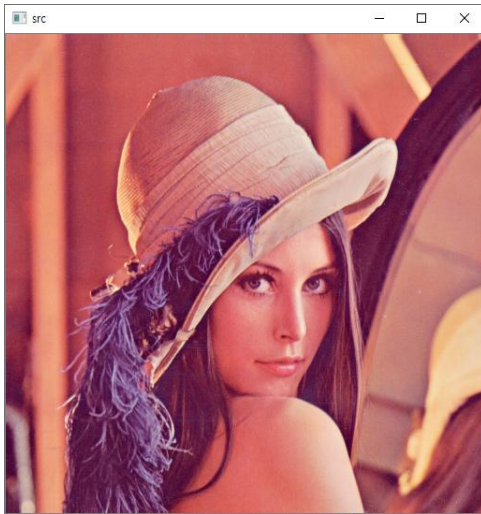
실습과제2

- 키보드로부터 회전각도를 입력받아 원하는 각도만큼 입력영상을 회전시키는 프로그램을 작성하시오.
- 무한히 반복하고 q를 누르면 종료하도록 하라.
- 회전변환을 반복할 때마다 왜곡현상이 발생함을 관찰하고 이유를 설명하라. 영상의 왜곡이 발생하지 않도록 해보시오.

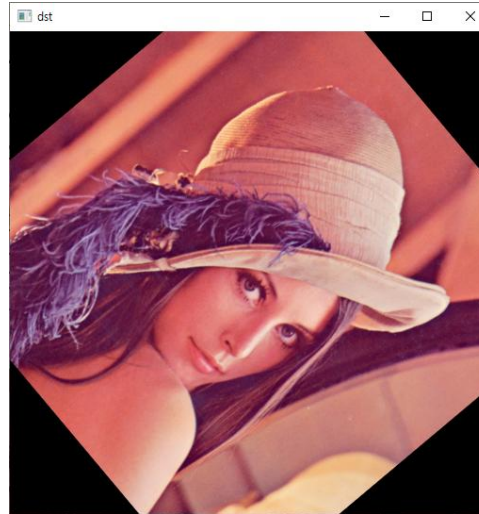


실습과제3

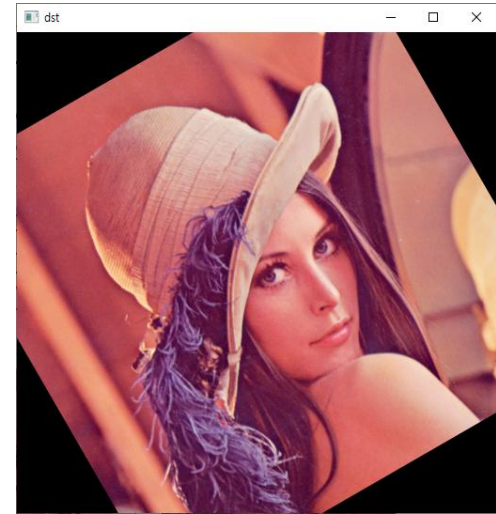
- 키보드에서 r키를 누를 때마다 영상의 중심좌표를 기준으로 시계방향으로 10도씩 회전하고 b키를 누를 때마다 반시계방향으로 10도씩 회전하도록 프로그램을 작성하시오. 예를 들어, r키 또는 b키를 36번 누르면 원본 영상으로 돌아와야 한다.
- 힌트 : 무한루프 안에서 키보드 입력값이 'r', 'b'일때만 회전각도를 조정하라. (키보드 이벤트 처리 방법은 157페이지 참고)



원본



원본영상에서 r키를
5번 누를때



원본영상에서 b키를
4번 누를때

실습과제4

- 코드 8-6의 예제를 flip함수 대신에 warpAffine함수를 이용하여 같은 결과가 나오도록 수정하시오.

실습과제5

- 코드 8-6의 예제에서 a키를 누르면 좌우대칭을 수행하고 b키를 누르면 상하대칭을 수행하고 c키를 누르면 좌우대칭과 상하대칭을 동시에 수행하도록 프로그램을 수정 하시오.
- 힌트 : 무한루프 안에서 키보드 입력값에 따라 다르게 처리하면 됨(키보드 이벤트 처리 방법은 157페이지 참고)

실습과제6

- 카메라로부터 입력 받은 동영상을 r을 누를 때마다 시계방향으로 90도씩 회전하도록 하시오. q를 누르면 종료하도록 하시오.
- 회전 변환을 반복할 때마다 영상의 왜곡이 생기는가? 이유를 설명하라.
- 반복문안에 waitKey함수는 1번만 호출할 것, 2번 이상 호출하면 0점 처리

과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출 하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.